

University Of Engineering and Technology, Lahore

Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab Report	Dated: July 1, 2026
Semester: 1st Semester	Session: Spring 2026
Lab/Project/Assignment #: 15	CLOs to be covered: CLO3
Lab Title: GUI using Qt Designer	Teacher Name: Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	Section: CYS-A

Lab Evaluation:

CLO3	Use appropriate data structures to organize, store, and manipulate data efficiently.					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
-------	-------	-------	--------

Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 15

Lab Goals / Objectives

- Understand the concept of Graphical User Interface (GUI) programming
- Learn about PyQt5 and its core components
- Understand Qt Designer for visual GUI design
- Learn about widgets, layouts, and signals/slots
- Convert .ui files to Python code using pyuic5
- Understand event-driven programming paradigm
- Create a functional GUI application

Equipment Required

- Computer System with Python 3.x installed
- PyQt5 library installed (pip install PyQt5)
- Qt Designer application
- Text Editor or IDE (PyCharm, VS Code, or IDLE)
- Operating System: Windows / Linux / macOS

Theory

1. What is GUI Programming?

Graphical User Interface (GUI) programming involves creating visual interfaces for applications that users can interact with using graphical elements like buttons, text fields, labels, and menus. Unlike command-line interfaces, GUIs provide a more intuitive and user-friendly experience.

2. Introduction to PyQt5

PyQt5 is a set of Python bindings for the Qt5 application framework. Qt is a comprehensive C++ library for developing cross-platform applications with GUIs. PyQt5 allows Python programmers to create complex GUI applications with ease.

3. Core PyQt5 Modules

- QtCore: Core non-GUI functionality (event loop, timers, data structures)
- QtGui: GUI components (fonts, colors, icons, images)
- QtWidgets: UI elements (buttons, labels, text fields, layouts)

4. Common PyQt5 Widgets

- QApplication, QWidget, QLabel, QPushButton, QLineEdit, QTextEdit
- QComboBox, QCheckBox, QRadioButton, QSlider, QProgressBar
- QMessageBox, QMainWindow, QTableWidget, QDialog, QTabWidget

5. Qt Designer

Qt Designer is a visual tool for designing GUIs. It allows dragging and dropping widgets onto a canvas, setting properties, and defining layouts. The designed interface is saved as a .ui file which can be converted to Python code.

9. The CoreStock Application

CoreStock is the GUI application built for this lab using PyQt5 and SQLite. It demonstrates practical application of GUI programming concepts including form design, data entry, table views, and interactive controls across a complete, multi-screen desktop workflow.

Project Proposal

Introduction

CoreStock is a desktop inventory and sales-floor management application built for a small-to-medium PC hardware retail store. It replaces manual, paper-based or spreadsheet-based stock tracking with a single, role-based desktop application that lets store staff manage products, record stock movements, maintain a supplier list, and generate stock reports — all from one consistent graphical interface.

The project was developed entirely in Python using the PyQt5 framework for the graphical interface and SQLite as the embedded database engine, and is version-controlled on GitHub.

Objectives

- Provide a secure, login-protected entry point so only authorized staff can access store data.
- Maintain a complete, searchable product catalogue with category, price, quantity, and a configurable low-stock alert threshold.
- Allow staff to record every stock movement and have product quantities update automatically as a result.

- Maintain a basic supplier directory for procurement contacts.
- Summarize stock activity in a reporting view, exportable as CSV.
- Present all of the above through a clean, modern, consistent PyQt5 interface with a custom stylesheet.

Scope

CoreStock is scoped as a single-store, single-machine desktop application rather than a networked, multi-branch system — there is no cloud sync, multi-user concurrency, or external API integration. Within this scope it fully supports user authentication, product inventory management, stock-in/stock-out tracking with automatic quantity adjustment, supplier record-keeping, and exportable stock reports.

Tools and Technologies Used

- Programming Language: Python 3
- GUI Framework: PyQt5 (QtWidgets, QtCore, QtGui)
- Database Engine: SQLite 3 (via Python's built-in sqlite3 module)
- IDE: PyCharm
- Version Control: Git and GitHub
- Architecture Style: Modular, single-window desktop app with tabbed navigation

Project Implementation — Source Code Walkthrough

The CoreStock PC Store Management System is organized into eight Python modules under a single package. Each module below is presented with a short description of its role followed by its complete source code.

main.py

Application entry point. Initializes the SQLite database, creates the QApplication instance, applies a global stylesheet (Segoe UI font, indigo accent color, rounded buttons and inputs), and launches the Login window.

```
import sys

from PyQt5.QtWidgets import QApplication

import database

from login import LoginWindow


database.setup()

app = QApplication(sys.argv)

app.setStyleSheet("""

    QWidget { font-family: Segoe UI; font-size: 13px; color: #1c1c1e; background: white; }

    QPushButton { background: #6366f1; color: white; border-radius: 7px; padding: 6px 14px; font-weight: bold; border: none; }

    QPushButton:hover { background: #4f46e5; }

    QLineEdit, QComboBox, QSpinBox, QDoubleSpinBox { border: 1px solid #e8e8ed; border-radius: 6px; padding: 5px 8px; background: #f8f8fa; }

    QLineEdit:focus { border: 1px solid #6366f1; background: white; }

    QTableWidget { background: white; border: 1px solid #e8e8ed; gridline-color: #f3f3f6; }

    QTableWidget::item:selected { background: #eef2ff; color: #4338ca; }

    QHeaderView::section { background: #f8f8fa; color: #6b6b6b; font-weight: bold; padding: 7px; border: none; border-bottom: 1px solid #e8e8ed; }

    QTabBar::tab { background: transparent; color: #a1a1aa; padding: 9px 20px; border-bottom: 2px solid transparent; }

    QTabBar::tab:selected { color: #6366f1; border-bottom: 2px solid #6366f1; font-weight: bold; }

    QTabWidget::pane { border: 1px solid #e8e8ed; }
```

```
        QDialog { background: white; }

    """

    window = LoginWindow()

    window.show()

    sys.exit(app.exec_())
```

database.py

Data access layer. Every database operation in the app goes through this module — creating the four tables (users, products, suppliers, movements) on first run, seeding two default accounts, and providing functions for login checks, product/supplier CRUD, low-stock lookups, and stock movement recording.

```
import sqlite3

def setup():

    con = sqlite3.connect("store.db")

    cur = con.cursor()

    cur.execute("CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY, username TEXT, password TEXT, role TEXT)")

    cur.execute("CREATE TABLE IF NOT EXISTS products (id INTEGER PRIMARY KEY, name TEXT, category TEXT, price REAL, quantity INTEGER, low_limit INTEGER)")

    cur.execute("CREATE TABLE IF NOT EXISTS suppliers (id INTEGER PRIMARY KEY, name TEXT, contact TEXT, email TEXT)")

    cur.execute("CREATE TABLE IF NOT EXISTS movements (id INTEGER PRIMARY KEY, product TEXT, type TEXT, qty INTEGER, date TEXT)")

    cur.execute("SELECT * FROM users WHERE username='admin'")

    if not cur.fetchone():

        cur.execute("INSERT INTO users VALUES (NULL, 'admin', 'admin123', 'Admin')")

        cur.execute("INSERT INTO users VALUES (NULL, 'staff', 'staff123', 'Staff')")

    con.commit()

    con.close()
```

```
def check_login(u, p):

    con = sqlite3.connect("store.db")

    cur = con.cursor()

    cur.execute("SELECT * FROM users WHERE username=? AND password=?", (u, p))

    row = cur.fetchone()

    con.close()

    return row


def get_products():

    con = sqlite3.connect("store.db")

    cur = con.cursor()

    cur.execute("SELECT * FROM products")

    rows = cur.fetchall()

    con.close()

    return rows


def add_product(name, cat, price, qty, low):

    con = sqlite3.connect("store.db")

    cur = con.cursor()

    cur.execute("INSERT INTO products VALUES (NULL,?,?,?,?,?)", (name, cat, price, qty, low))

    con.commit()

    con.close()


def update_product(pid, name, cat, price, qty, low):

    con = sqlite3.connect("store.db")

    cur = con.cursor()

    cur.execute("UPDATE products SET name=?,category=?,price=?,quantity=?,low_limit=? WHERE id=?",
(name, cat, price, qty, low, pid))
```

```
con.commit()

con.close()

def delete_product(pid):

    con = sqlite3.connect("store.db")

    cur = con.cursor()

    cur.execute("DELETE FROM products WHERE id=?", (pid,))

    con.commit()

    con.close()

def get_low_stock():

    con = sqlite3.connect("store.db")

    cur = con.cursor()

    cur.execute("SELECT * FROM products WHERE quantity <= low_limit")

    rows = cur.fetchall()

    con.close()

    return rows

def get_suppliers():

    con = sqlite3.connect("store.db")

    cur = con.cursor()

    cur.execute("SELECT * FROM suppliers")

    rows = cur.fetchall()

    con.close()

    return rows

def add_supplier(name, contact, email):

    con = sqlite3.connect("store.db")
```



```

cur = con.cursor()

cur.execute("INSERT INTO suppliers VALUES (NULL,?,?,?)", (name, contact, email))

con.commit()

con.close()


def get_movements():

    con = sqlite3.connect("store.db")

    cur = con.cursor()

    cur.execute("SELECT * FROM movements")

    rows = cur.fetchall()

    con.close()

    return rows


def add_movement(product, mtype, qty, date):

    con = sqlite3.connect("store.db")

    cur = con.cursor()

    cur.execute("INSERT INTO movements VALUES (NULL,?,?,?,?)", (product, mtype, qty, date))

    if mtype == "IN":

        cur.execute("UPDATE products SET quantity=quantity+? WHERE name=?", (qty, product))

    else:

        cur.execute("UPDATE products SET quantity=quantity-? WHERE name=?", (qty, product))

    con.commit()

    con.close()

```

login.py

Authentication screen. A fixed-size window with the CoreStock logo, username/password fields, and a login button. Verifies credentials against the users table and, on success, opens the main Dashboard while closing the login window.

```
from PyQt5.QtWidgets import QWidget, QVBoxLayout, QLabel, QLineEdit, QPushButton, QMessageBox,
QFrame

from PyQt5.QtCore import Qt

from PyQt5.QtGui import QFont

import database


class LoginWindow(QWidget):

    def __init__(self):
        super().__init__()

        self.setWindowTitle("CoreStock")

        self.setFixedSize(380, 450)

        self.setStyleSheet("background: white;")


        layout = QVBoxLayout()

        layout.setContentsMargins(45, 35, 45, 35)

        layout.setSpacing(10)


        logo = QLabel("C")

        logo.setFixedSize(50, 50)

        logo.setAlignment(Qt.AlignCenter)

        logo.setFont(QFont("Segoe UI", 20, QFont.Bold))

        logo.setStyleSheet("background:#6366f1; color:white; border-radius:12px;")


        title = QLabel("CoreStock")

        title.setAlignment(Qt.AlignCenter)

        title.setFont(QFont("Segoe UI", 20, QFont.Bold))


        sub = QLabel("PC Store Management System")
```

```
sub.setAlignment(Qt.AlignCenter)

sub.setStyleSheet("color:#a1a1aa; font-size:12px;")


line = QFrame()
line.setFrameShape(QFrame.HLine)
line.setStyleSheet("background:#e8e8ed;")
line.setFixedHeight(1)


self.username = QLineEdit()
self.username.setPlaceholderText("Username")
self.username.setFixedHeight(40)


self.password = QLineEdit()
self.password.setPlaceholderText("Password")
self.password.setEchoMode(QLineEdit.Password)
self.password.setFixedHeight(40)


btn = QPushButton("Login")
btn.setFixedHeight(42)
btn.setStyleSheet("background:#6366f1; color:white; border-radius:7px; font-size:13px; font-weight:bold; border:none;")
btn.clicked.connect(self.do_login)


hint = QLabel("admin / admin123")
hint.setAlignment(Qt.AlignCenter)
hint.setStyleSheet("color:#ccc; font-size:11px;")


layout.addWidget(logo, alignment=Qt.AlignCenter)
```

```

        layout.addWidget(title)

        layout.addWidget(sub)

        layout.addWidget(line)

        layout.addSpacing(5)

        layout.addWidget(self.username)

        layout.addWidget(self.password)

        layout.addSpacing(8)

        layout.addWidget(btn)

        layout.addWidget(hint)

        self.setLayout(layout)

def do_login(self):

    u = self.username.text().strip()

    p = self.password.text().strip()

    user = database.check_login(u, p)

    if user:

        from dashboard import Dashboard

        self.dash = Dashboard(user)

        self.dash.show()

        self.close()

    else:

        QMessageBox.warning(self, "Error", "Wrong username or password!")

```

dashboard.py

Main application shell. Builds the dark header bar, the four live summary cards (Products, Total Units, Low Stock, Suppliers), and a tabbed interface hosting the Products, Stock In/Out, Suppliers, and Reports pages. Also handles logout.

```

from PyQt5.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QLabel, QPushButton, QTabWidget,
QFrame

```

```
from PyQt5.QtGui import QFont

from PyQt5.QtCore import Qt

import database


class Dashboard(QWidget):

    def __init__(self, user):

        super().__init__()

        self.user = user

        self.setWindowTitle("CoreStock")

        self.setMinimumSize(900, 560)


        layout = QVBoxLayout()

        layout.setContentsMargins(0, 0, 0, 0)

        layout.setSpacing(0)


        # header

        header = QFrame()

        header.setFixedHeight(54)

        header.setStyleSheet("background:#1c1c1e;")

        h = QHBoxLayout(header)

        h.setContentsMargins(16, 0, 16, 0)


        logo = QLabel("C")

        logo.setFixedSize(30, 30)

        logo.setAlignment(Qt.AlignCenter)

        logo.setFont(QFont("Segoe UI", 13, QFont.Bold))

        logo.setStyleSheet("background:#6366f1; color:white; border-radius:8px;")
```

```
name = QLabel("CoreStock")

name.setFont(QFont("Segoe UI", 13, QFont.Bold))

name.setStyleSheet("color:white;")


sub = QLabel("PC Store Management")

sub.setStyleSheet("color:#555; font-size:11px;")


who = QLabel(user[1] + " | " + user[3])

who.setStyleSheet("color:#888; font-size:11px;")


out = QPushButton("Logout")

out.setFixedSize(68, 26)

out.setStyleSheet("background:#6366f1; color:white; border-radius:6px; font-size:11px;
border:none;")

out.clicked.connect(self.logout)


h.addWidget(logo)

h.addSpacing(8)

h.addWidget(name)

h.addSpacing(6)

h.addWidget(sub)

h.addStretch()

h.addWidget(who)

h.addSpacing(10)

h.addWidget(out)


# stats

stats = QFrame()
```

```

stats.setFixedHeight(84)

stats.setStyleSheet("background:#f8f8fa; border-bottom:1px solid #e8e8ed;")

s = QHBoxLayout(stats)

s.setContentsMargins(16, 10, 16, 10)

s.setSpacing(10)


prods = database.get_products()

low    = database.get_low_stock()

sups   = database.get_suppliers()

units = sum(p[4] for p in prods)


for title, val, color in [

    ("Products",    len(prods), "#6366f1"),

    ("Total Units", units,      "#1c1c1e"),

    ("Low Stock",   len(low),   "#ef4444"),

    ("Suppliers",   len(sups),  "#1c1c1e"),

]:

    card = QFrame()

    card.setStyleSheet("background:white; border-radius:9px; border:1px solid #e8e8ed;")

    cl = QVBoxLayout(card)

    cl.setContentsMargins(12, 6, 12, 6)

    n = QLabel(str(val))

    n.setFont(QFont("Segoe UI", 18, QFont.Bold))

    n.setStyleSheet(f"color:{color};")

    l = QLabel(title)

    l.setStyleSheet("color:#aaa; font-size:10px;")

    cl.addWidget(n)

    cl.addWidget(l)

    s.addWidget(card)

```

```

# tabs

tabs = QTabWidget()

from products import ProductsPage

from stock import StockPage

from suppliers import SuppliersPage

from reports import ReportsPage

tabs.addTab(ProductsPage(), "Products")

tabs.addTab(StockPage(), "Stock In/Out")

tabs.addTab(SuppliersPage(), "Suppliers")

tabs.addTab(ReportsPage(), "Reports")


layout.addWidget(header)

layout.addWidget(stats)

layout.addWidget(tabs)

self.setLayout(layout)


def logout(self):

    from login import LoginWindow

    self.w = LoginWindow()

    self.w.show()

    self.close()

```

products.py

Inventory (Products) module. Displays all products in a searchable table, with Add/Edit/Delete actions through a shared PForm dialog. Includes a live low-stock alert banner refreshed via `refresh_alert()` every time the table reloads.

```

from PyQt5.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QLabel, QPushButton,

```



```

        QLineEdit, QTableWidgetItem, QMessageBox,

        QDialog, QFormLayout, QComboBox, QSpinBox, QDoubleSpinBox)

from PyQt5.QtCore import Qt

import database


class ProductsPage(QWidget):

    def __init__(self):
        super().__init__()

        layout = QVBoxLayout()

        layout.setContentsMargins(14, 14, 14, 14)

        top = QHBoxLayout()

        top.addWidget(QLabel("Products"))

        top.addStretch()

        a = QPushButton("+ Add Product")

        a.clicked.connect(self.add)

        top.addWidget(a)

        self.alert = QLabel()

        self.search = QLineEdit()

        self.search.setPlaceholderText("Search...")

        self.search.textChanged.connect(self.do_search)

        self.table = QTableWidgetItem()

        self.table.setColumnCount(6)

        self.table.setHorizontalHeaderLabels(["ID", "Name", "Category", "Price", "Qty", "Low Limit"])

        self.table.setEditTriggers(QTableWidgetItem.NoEditTriggers)

        self.table.setSelectionBehavior(QTableWidgetItem.SelectRows)

```

```

self.table.verticalHeader().setVisible(False)

self.table.horizontalHeader().setStretchLastSection(True)


bot = QHBoxLayout()
bot.addStretch()

e = QPushButton("Edit")
e.clicked.connect(self.edit)

d = QPushButton("Delete")

d.setStyleSheet("background:#ef4444; color:white; border-radius:7px; padding:6px 14px; font-
weight:bold; border:none;")

d.clicked.connect(self.delete)

bot.addWidget(e)
bot.addWidget(d)


layout.addLayout(top)
layout.addWidget(self.alert)
layout.addWidget(self.search)
layout.addWidget(self.table)
layout.addLayout(bot)

self.setLayout(layout)

self.load()


def refresh_alert(self):

    low = database.get_low_stock()

    if low:

        self.alert.setText("⚠ Low Stock: " + ", ".join(p[1] for p in low))

        self.alert.setStyleSheet("background:#fef2f2; color:#dc2626; border:1px solid #fca5a5;
border-radius:6px; padding:7px;")

    else:

```

```

        self.alert.setText("✓ All stock levels are fine")

        self.alert.setStyleSheet("background:#f0fdf4; color:#16a34a; border:1px solid #86efac;
border-radius:6px; padding:7px;")

def load(self):

    self.refresh_alert()

    self.fill(database.get_products())

def fill(self, data):

    self.table.setRowCount(0)

    for row in data:

        r = self.table.rowCount()

        self.table.insertRow(r)

        for c, v in enumerate(row):

            i = QTableWidgetItem(str(v or ""))

            i.setTextAlignment(Qt.AlignCenter)

            self.table.setItem(r, c, i)

def do_search(self):

    w = self.search.text().lower()

    self.fill([p for p in database.get_products() if w in p[1].lower() or w in (p[2] or
"").lower()])

def add(self):

    d = PForm(self)

    if d.exec_(): self.load()

def edit(self):

    row = self.table.currentRow()

```

```

        if row == -1:

            QMessageBox.warning(self, "!", "Select a product first!"); return

        data = [self.table.item(row, i).text() for i in range(6)]

        d = PForm(self, data)

        if d.exec_(): self.load()

def delete(self):

    row = self.table.currentRow()

    if row == -1:

        QMessageBox.warning(self, "!", "Select a product first!"); return

    pid = int(self.table.item(row, 0).text())

    name = self.table.item(row, 1).text()

    if QMessageBox.question(self, "Delete?", f"Delete '{name}'?") == QMessageBox.Yes:

        database.delete_product(pid)

        self.load()

class PForm(QDialog):

    def __init__(self, parent, data=None):

        super().__init__(parent)

        self.data = data

        self.setWindowTitle("Add Product" if not data else "Edit Product")

        self.setFixedSize(320, 290)

        self.setStyleSheet("background:white;")

        layout = QFormLayout()

        layout.setContentsMargins(20, 20, 20, 20)

        layout.setSpacing(10)

        self.name = QLineEdit()

        self.cat = QComboBox()

```

```

self.cat.addItem(["GPU", "CPU", "RAM", "Monitor", "Storage", "Motherboard", "Keyboard", "Mouse", "Other"])

self.price = QDoubleSpinBox()
self.price.setMaximum(999999)

self.qty = QSpinBox()
self.qty.setMaximum(99999)

self.low = QSpinBox()
self.low.setValue(5)

if data:

    self.name.setText(data[1])

    self.cat.setCurrentText(data[2])

    self.price.setValue(float(data[3] or 0))

    self.qty.setValue(int(data[4] or 0))

    self.low.setValue(int(data[5] or 5))

layout.addRow("Name:", self.name)
layout.addRow("Category:", self.cat)
layout.addRow("Price:", self.price)
layout.addRow("Quantity:", self.qty)
layout.addRow("Low Alert:", self.low)

s = QPushButton("Save")
s.clicked.connect(self.save)

layout.addRow(s)

self.setLayout(layout)

def save(self):

    if not self.name.text().strip():

        QMessageBox.warning(self, "!", "Name is required!"); return

    if self.data:

        database.update_product(int(self.data[0]), self.name.text(), self.cat.currentText(),
self.price.value(), self.qty.value(), self.low.value())

```

```
        else:

            database.add_product(self.name.text(), self.cat.currentText(), self.price.value(),
self.qty.value(), self.low.value())

        self.accept()
```

stock.py

Stock In/Out module. Lets staff record incoming or outgoing stock movements through a dialog, automatically adjusting product quantities in the database and blocking an OUT movement if it would take stock negative.

```
from PyQt5.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QLabel, QPushButton,
                             QTableWidgetItem, QTableWidgetItem, QDialog, QFormLayout,
                             QComboBox, QSpinBox, QMessageBox)

from PyQt5.QtCore import Qt

from datetime import date

import database

class StockPage(QWidget):

    def __init__(self):

        super().__init__()

        layout = QVBoxLayout()

        layout.setContentsMargins(14, 14, 14, 14)

        top = QHBoxLayout()

        top.addWidget(QLabel("Stock In / Out"))

        top.addStretch()

        b = QPushButton("+ Record Movement")

        b.clicked.connect(self.record)

        top.addWidget(b)
```

```

self.alert = QLabel()

self.table = QTableWidgetItem()

self.table.setColumnCount(5)

self.table.setHorizontalHeaderLabels(["ID", "Product", "Type", "Qty", "Date"])

self.table.setEditTriggers(QTableWidgetItem.NoEditTriggers)

self.table.verticalHeader().setVisible(False)

self.table.horizontalHeader().setStretchLastSection(True)

layout.addLayout(top)

layout.addWidget(self.alert)

layout.addWidget(self.table)

self.setLayout(layout)

self.load()

def refresh_alert(self):

    low = database.get_low_stock()

    if low:

        self.alert.setText("⚠ Low Stock: " + ", ".join(p[1] for p in low))

        self.alert.setStyleSheet("background:#fef2f2; color:#dc2626; border:1px solid #fca5a5;
border-radius:6px; padding:7px;")

    else:

        self.alert.setText("✓ All stock levels are fine")

        self.alert.setStyleSheet("background:#f0fdf4; color:#16a34a; border:1px solid #86efac;
border-radius:6px; padding:7px;")

def load(self):

    self.refresh_alert()

    data = database.get_movements()

    self.table.setRowCount(0)

    for row in data:

```

```

        r = self.table.rowCount()

        self.table.insertRow(r)

        for c, v in enumerate(row):

            i = QTableWidgetItem(str(v or ""))

            i.setTextAlignment(Qt.AlignCenter)

            self.table.setItem(r, c, i)

    def record(self):

        d = SForm(self)

        if d.exec_(): self.load()

class SForm(QDialog):

    def __init__(self, parent):

        super().__init__(parent)

        self.setWindowTitle("Record Movement")

        self.setFixedSize(300, 200)

        self.setStyleSheet("background:white;")

        layout = QFormLayout()

        layout.setContentsMargins(20, 20, 20, 20)

        layout.setSpacing(10)

        self.product = QComboBox()

        self.products = database.get_products()

        for p in self.products:

            self.product.addItem(f"{p[1]} (qty:{p[4]})", p[1])

        self.mtype = QComboBox()

        self.mtype.addItem("IN", "OUT")

        self.qty = QSpinBox()

        self.qty.setMinimum(1)

        self.qty.setMaximum(9999)

```



```

s = QPushButton("Save")

s.clicked.connect(self.save)

layout.addRow("Product:", self.product)

layout.addRow("Type:", self.mtype)

layout.addRow("Qty:", self.qty)

layout.addRow(s)

self.setLayout(layout)

def save(self):
    name = self.product.currentData()

    move = self.mtype.currentText()

    qty = self.qty.value()

    if move == "OUT":
        p = next((x for x in self.products if x[1] == name), None)

        if p and p[4] < qty:
            QMessageBox.warning(self, "!", f"Only {p[4]} in stock!"); return

        database.add_movement(name, move, qty, str(date.today()))

    self.accept()

```

suppliers.py

Supplier directory module. A simple CRUD screen for maintaining supplier name, contact number, and email address, used for procurement reference.

```

from PyQt5.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QLabel, QPushButton,
                             QTableWidgetItem, QTableWidgetItem, QDialog, QFormLayout,
                             QLineEdit, QMessageBox)

from PyQt5.QtCore import Qt

import database

```

```

class SuppliersPage(QWidget):
    def __init__(self):
        super().__init__()

        layout = QVBoxLayout()

        layout.setContentsMargins(14, 14, 14, 14)

        top = QHBoxLayout()

        top.addWidget(QLabel("Suppliers"))

        top.addStretch()

        b = QPushButton("+ Add Supplier")

        b.clicked.connect(self.add)

        top.addWidget(b)

        self.table = QTableWidget()

        self.table.setColumnCount(4)

        self.table.setHorizontalHeaderLabels(["ID", "Name", "Contact", "Email"])

        self.table.setEditTriggers(QTableWidget.NoEditTriggers)

        self.table.verticalHeader().setVisible(False)

        self.table.horizontalHeader().setStretchLastSection(True)

        layout.addLayout(top)

        layout.addWidget(self.table)

        self.setLayout(layout)

        self.load()

    def load(self):
        data = database.get_suppliers()

        self.table.setRowCount(0)

        for row in data:
            r = self.table.rowCount()

            self.table.insertRow(r)

```

```

        for c, v in enumerate(row):

            i = QTableWidgetItem(str(v or ""))

            i.setTextAlignment(Qt.AlignCenter)

            self.table.setItem(r, c, i)

def add(self):

    d = SupForm(self)

    if d.exec_(): self.load()

class SupForm(QDialog):

    def __init__(self, parent):

        super().__init__(parent)

        self.setWindowTitle("Add Supplier")

        self.setFixedSize(300, 190)

        self.setStyleSheet("background:white;")

        layout = QFormLayout()

        layout.setContentsMargins(20, 20, 20, 20)

        layout.setSpacing(10)

        self.name = QLineEdit()

        self.contact = QLineEdit()

        self.email = QLineEdit()

        s = QPushButton("Save")

        s.clicked.connect(self.save)

        layout.addRow("Name:", self.name)

        layout.addRow("Contact:", self.contact)

        layout.addRow("Email:", self.email)

        layout.addRow(s)

        self.setLayout(layout)

```

```

def save(self):

    if not self.name.text().strip():

        QMessageBox.warning(self, "!", "Name required!"); return

    database.add_supplier(self.name.text(), self.contact.text(), self.email.text())

    self.accept()

```

reports.py

Reporting module. Summarizes total stock IN/OUT and transaction counts, displays the full movement history in a table, and allows the report to be exported as a CSV file.

```

from PyQt5.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QLabel, QPushButton,
                             QTableWidgetItem, QFileDialog, QMessageBox)

from PyQt5.QtCore import Qt

import database

class ReportsPage(QWidget):

    def __init__(self):
        super().__init__()

        layout = QVBoxLayout()

        layout.setContentsMargins(14, 14, 14, 14)

        top = QHBoxLayout()

        top.addWidget(QLabel("Stock Report"))

        top.addStretch()

        e = QPushButton("Export CSV")

        e.clicked.connect(self.export)

        top.addWidget(e)

        data = database.get_movements()

        tin = sum(m[3] for m in data if m[2] == "IN")

```

```

        tout = sum(m[3] for m in data if m[2] == "OUT")

        summary = QLabel(f"Total IN: {tin} | Total OUT: {tout} | Transactions: {len(data)}")

        summary.setStyleSheet("background:#f8f8fa; border:1px solid #e8e8ed; border-radius:6px;
padding:8px; color:#444;")

        self.table = QTableWidgetItem()

        self.table.setColumnCount(5)

        self.table.setHorizontalHeaderLabels(["ID", "Product", "Type", "Qty", "Date"])

        self.table.setEditTriggers(QTableWidgetItem.NoEditTriggers)

        self.table.verticalHeader().setVisible(False)

        self.table.horizontalHeader().setStretchLastSection(True)

        layout.addLayout(top)

        layout.addWidget(summary)

        layout.addWidget(self.table)

        self.setLayout(layout)

        self.load()

def load(self):

    data = database.get_movements()

    self.table.setRowCount(0)

    for row in data:

        r = self.table.rowCount()

        self.table.insertRow(r)

        for c, v in enumerate(row):

            i = QTableWidgetItem(str(v or ""))

            i.setTextAlignment(Qt.AlignCenter)

            self.table.setItem(r, c, i)

def export(self):

    path, _ = QFileDialog.getSaveFileName(self, "Save", "report.csv", "CSV (*.csv)")

```

```
if path:

    data = database.get_movements()

    with open(path, "w") as f:

        f.write("ID,Product,Type,Qty,Date\n")

        for row in data:

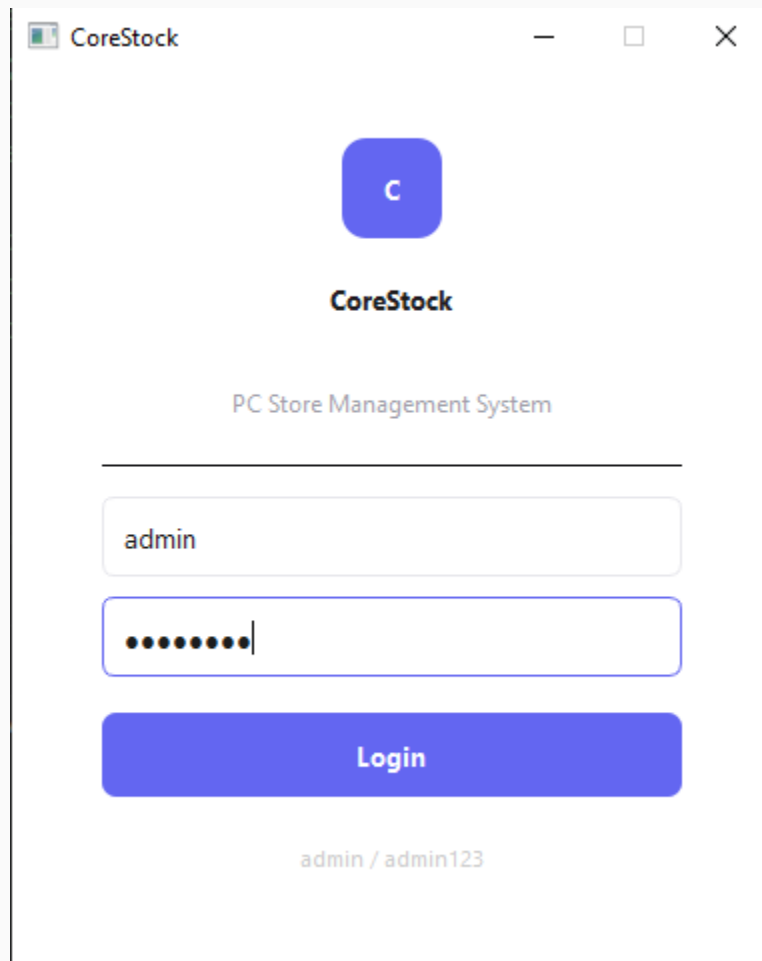
            f.write(",".join(str(x or "") for x in row) + "\n")

    QMessageBox.information(self, "Done", "Saved!")
```

Application Output

Screenshots of the running CoreStock application, one for each screen of the workflow.

Login Screen



CoreStock

C

CoreStock

PC Store Management System

admin

••••••••

Login

admin / admin123

Dashboard — Overview & Summary Cards

CoreStock

admin | Admin

Logout

13

Products

132

Total Units

1

Low Stock

8

Suppliers

Products

Stock In/Out

Suppliers

Reports

Products

+ Add Product

⚠ Low Stock: ASUS ROG X870-E

Search...

ID	Name	Category	Price	Qty	Low Limit
1	RTX 4090	GPU	400000.0	9	3
2	RTX 5090	GPU	850000.0	9	2
3	Ryzen 7 ...	CPU	165000.0	14	4
4	Core i7-14700K	CPU	140000.0	10	3
5	Corsair ...	RAM	32000.0	17	5
6	ASUS ROG ...	Motherboard	115000.0	2	3
7	MSI B650M ...	Motherboard	62000.0	11	4
8	Dell G2724D	Monitor	85000.0	10	2
9	Samsung 990 ...	Storage	48000.0	15	4
10	DeepCool ...	Other	18000.0	9	3
11	MSI MAG ...	Monitor	55000.0	10	3
12	MSI G274QPF-...	Monitor	98000.0	7	4
13	I7 4790K	CPU	40000.0	9	5

Edit

Delete

Products Tab — Inventory List

CoreStock

admin | Admin

Logout

13

Products

132

Total Units

1

Low Stock

8

Suppliers

Products

Stock In/Out

Suppliers

Reports

Products

+ Add Product

⚠ Low Stock: ASUS ROG X870-E

Search...

ID	Name	Category	Price	Qty	Low Limit
1	RTX 4090	GPU	400000.0	9	3
2	RTX 5090	GPU	850000.0	9	2
3	Ryzen 7 ...	CPU	165000.0	14	4
4	Core i7-14700K	CPU	140000.0	10	3
5	Corsair ...	RAM	32000.0	17	5
6	ASUS ROG ...	Motherboard	115000.0	2	3
7	MSI B650M ...	Motherboard	62000.0	11	4
8	Dell G2724D	Monitor	85000.0	10	2
9	Samsung 990 ...	Storage	48000.0	15	4
10	DeepCool ...	Other	18000.0	9	3
11	MSI MAG ...	Monitor	55000.0	10	3
12	MSI G274QPF-...	Monitor	98000.0	7	4
13	I7 4790K	CPU	40000.0	9	5

Edit

Delete

Stock In/Out Tab:

CoreStock

C

CoreStock

PC Store Management

admin | Admin

Logout

13

Products

132

Total Units

1

Low Stock

8

Suppliers

Products

Stock In/Out

Suppliers

Reports

Stock In / Out

+ Record Movement

Low Stock: ASUS ROG X870-E

Suppliers Tab

CoreStock

C

CoreStock

PC Store Management

admin | Admin

Logout

13

Products

132

Total Units

1

Low Stock

8

Suppliers

Products

Stock In/Out

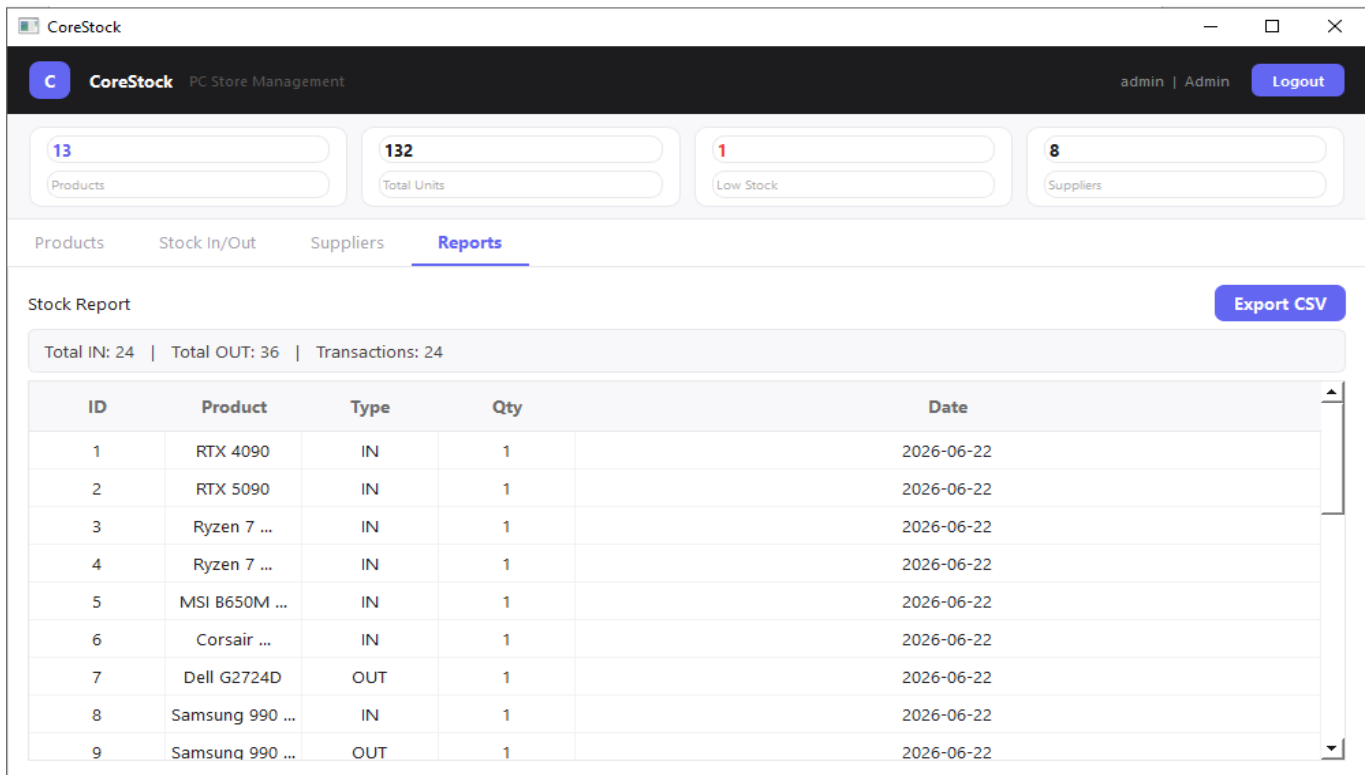
Suppliers

Reports

Suppliers

+ Add Supplier

Reports Tab



Conclusions

- Successfully understood the concept of GUI programming with PyQt5
- Learned about Qt Designer and how .ui files relate to Python code
- Understood the core PyQt5 modules: QtCore, QtGui, and QtWidgets
- Mastered the signals and slots mechanism for event handling
- Understood event-driven programming and the main event loop
- Designed and implemented a full relational schema in SQLite for products, suppliers, users, and stock movements
- Built and delivered CoreStock, a complete multi-screen PyQt5 + SQLite inventory management application, as the semester capstone project